CS3230 Lecture 11

Greedy Algorithms

(ksung@comp.nus.edu.sg)

Lecture Topics and Readings

- An activity-selection problem [CLRS] Ch16.1
- Huffman code [CLRS] Ch16.3
- Greedy Nearest Neighbor *heuristic* for TSP

For some problems, locally optimal is the same as globally optimal.

David Albert Huffman

He created the Huffman code
when he studied PhD in MIT

- D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes", **<u>1952</u>**

The idea is created when Huffman tried to work on a term paper assigned by Prof. Robert M. Fano

- "Find the most efficient method to represent numbers or symbols using binary code"

Many data communication devices/media
use Huffman code, e.g. FAX, JPEG, MP3

Greedy Algorithms

A (Re)-introduction

(You have seen Greedy Algorithms in Prim's, Kruskal's, and/or Dijkstra's algorithm before)

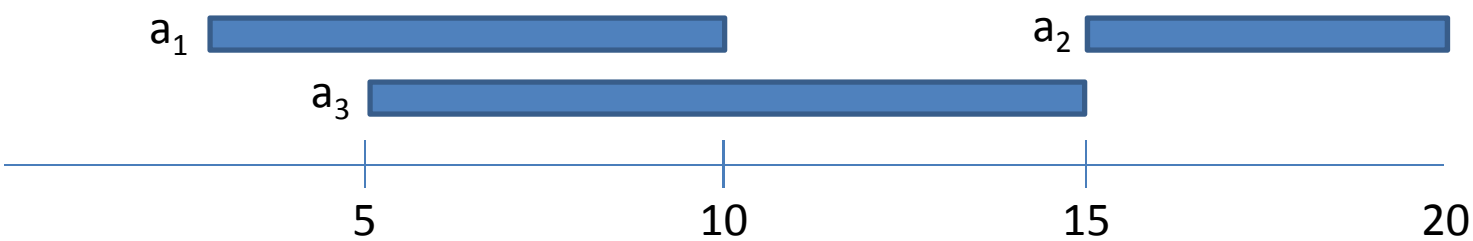
Activity-Selection problem (1)

You have a set of activities $S = \{a_1, a_2, \ldots, a_n\}$

- Each activity a_i takes place during $[s_i , f_i)$
- Two activities are compatible if $[s_i, f_i)$ and $[s_j, f_j)$ do not overlap, i.e., $s_i \geq f_j$ or $s_j \geq f_i$

Example: $a_1=[3, 10)$, $a_2=[15, 20)$, $a_3=[5, 15)$

- $\{a_1$ and $a_2\}$ and $\{a_2$ and $a_3\}$ are compatible
- $\{a_1$ and $a_3\}$ are not compatible

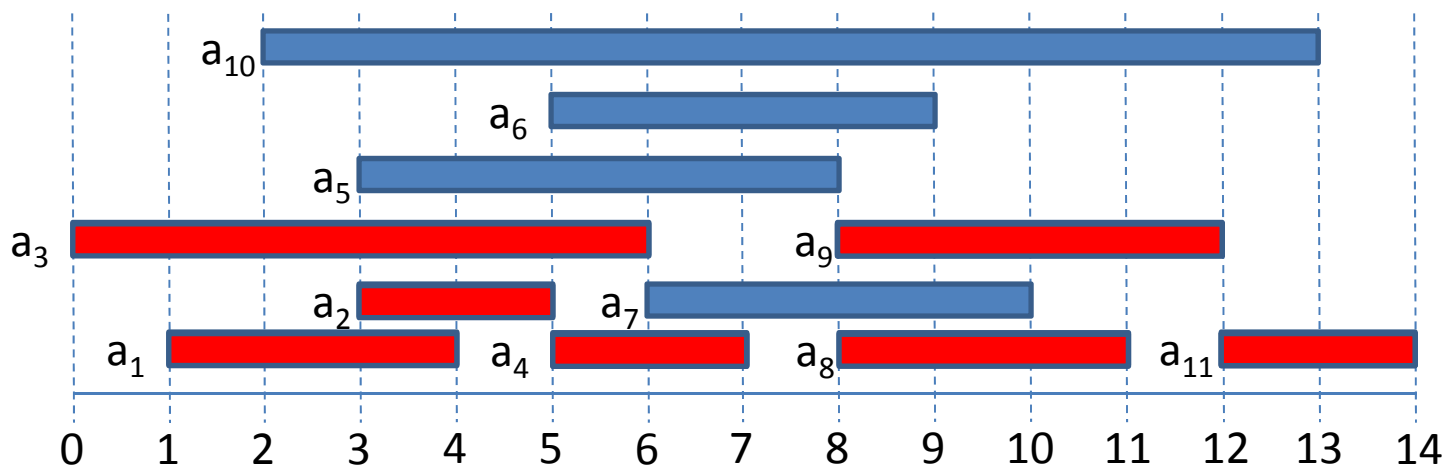


Activity-Selection problem (2)

Find a max-size subset of mutually compatible activities

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

- $\{a_3, a_9, a_{11}\}$ is a set of mutually compatible activities
- $\{a_1, a_4, a_8, a_{11}\}$ is a largest set of mutually compatible activities
- This is not unique, another largest set is $\{a_2, a_4, a_9, a_{11}\}$

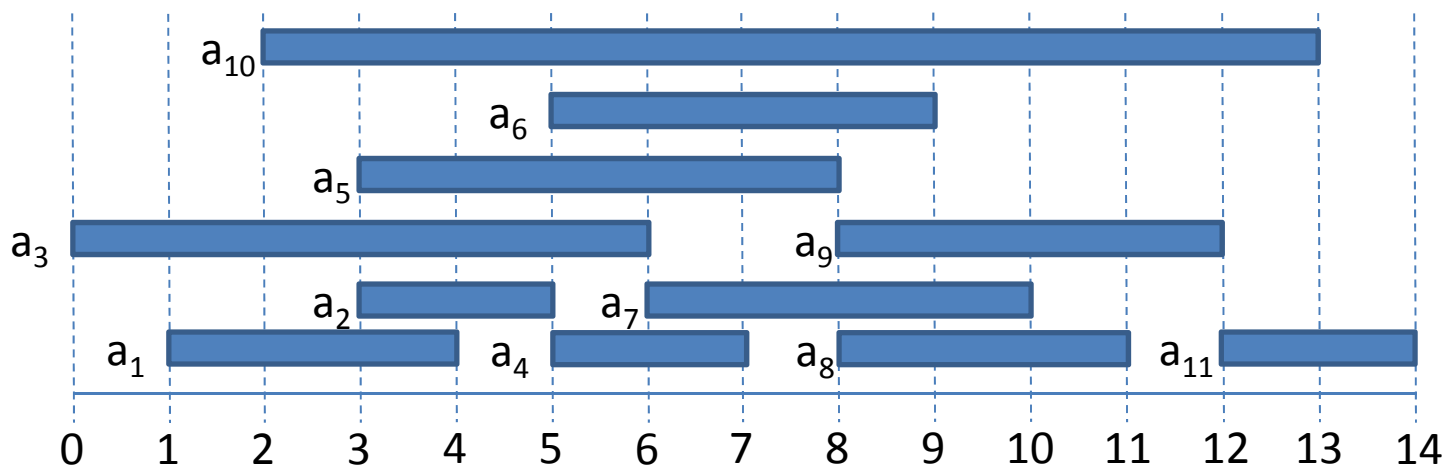


Optimal Substructure (1)

Let S_{ij} be the subset of activities $a_k \in S$ that start after a_i finishes and finish before a_j starts, i.e. $S_{ij} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$. Examples:

- $S_{0,12} = \{a_1, a_2, \ldots, a_{11}\}$
- $S_{1,11} = \{a_4, a_6, a_7, a_8, a_9\}$
- $S_{7,11} = \Phi$

i	0	1	2	3	4	5	6	7	8	9	10	11	12
s_i	0	1	3	0	5	3	5	6	8	8	2	12	∞
f_i	0	4	5	6	7	8	9	10	11	12	13	14	∞

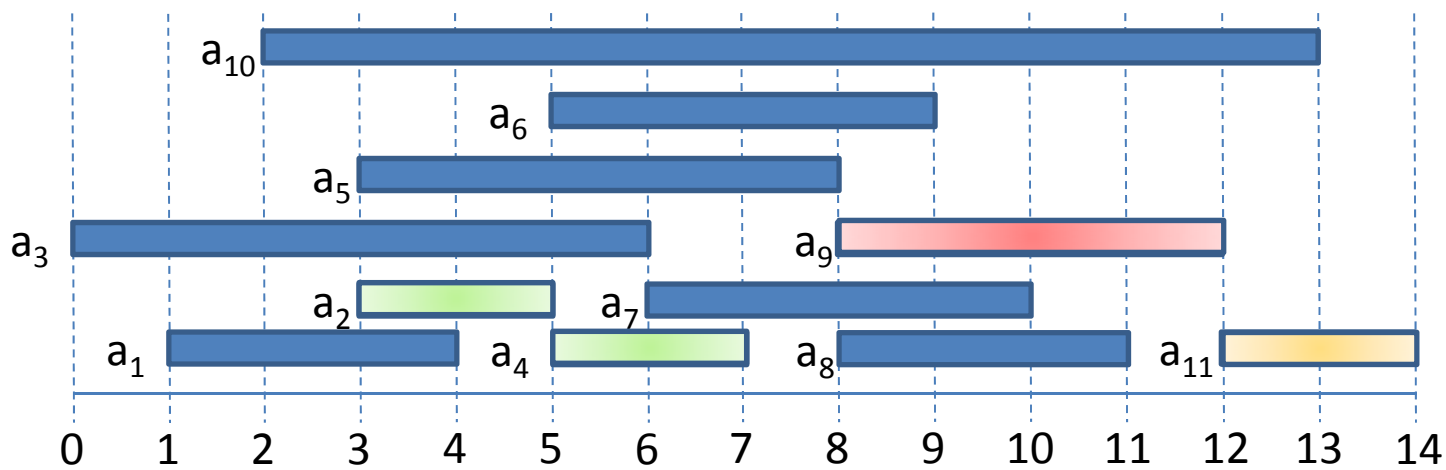


Optimal Substructure (2)

Let B_{ij} be an optimal solution for S_{ij}; Suppose $a_k \in B_{ij}$, then, $B_{ik} \cup \{a_k\} \cup B_{kj}$ is also an optimal solution for S_{ij}

Example: *Suppose* we know $a_9 \in B_{0,12}$

- $S_{0,9}=\{a_1,a_2,a_3,a_4,a_5\}$ and $B_{0,9}=\{a_2, a_4\}$
- $S_{9,12}=\{a_{11}\}$ and $B_{9,12}=\{a_{11}\}$
- Then, $B_{0,9} \cup \{a_9\} \cup B_{9,12}=\{a_2, a_4, a_9, a_{11}\}$, which is an optimal solution for $S_{0,12}$



Recursive Formula

$|B_{ij}| = \max_{a_k \in S_{ij}} |B_{ik} \cup \{a_k\} \cup B_{kj}|$, i.e. we try all a_k

Let c[i,j] = $|B_{ij}|$ = the maximum number of mutually compatible activities in S_{ij}

Base case: c[i,i] = 0

Recursive case: $c[i,j] =$

$$max \begin{cases} 0 & if\ S_{ij} = \emptyset \\ \max_{i<k<j, a_k \in S_{ij}} \{c[i,k] + c[k,j] + 1\} & if\ S_{ij} \neq \emptyset \end{cases}$$

Question: This is $O(n^3)$ with DP, can we avoid trying all k?

Making a Greedy Choice

DP solves **all the subproblems first** before it obtains the optimal solution for this problem

Can we avoid solving all subproblems?

Greedy-choice property
*A locally optimal choice is globally optimal**

* Terms and conditions apply: Only applicable for some problems

Greedy Choice

To compute $B_{0,n+1}$, we need to find a_k that appears in the optimal solution

- Then, $B_{0,k} \cup \{a_k\} \cup B_{k,n+1}$ is an optimal solution for $S_{0,n+1}$

Can we make a greedy choice?

- Intuition: The activity a_k in $S_{0,n+1}$ with the earliest finishing time is likely to be in the optimal solution

Standard Proof Technique

How to show that the greedy choice is good?

- In this case, the activity a_k in $S_{0,n+1}$ with the earliest finishing time is likely to be in the optimal solution

<u>Usually</u>, we do the following (exchange argument):

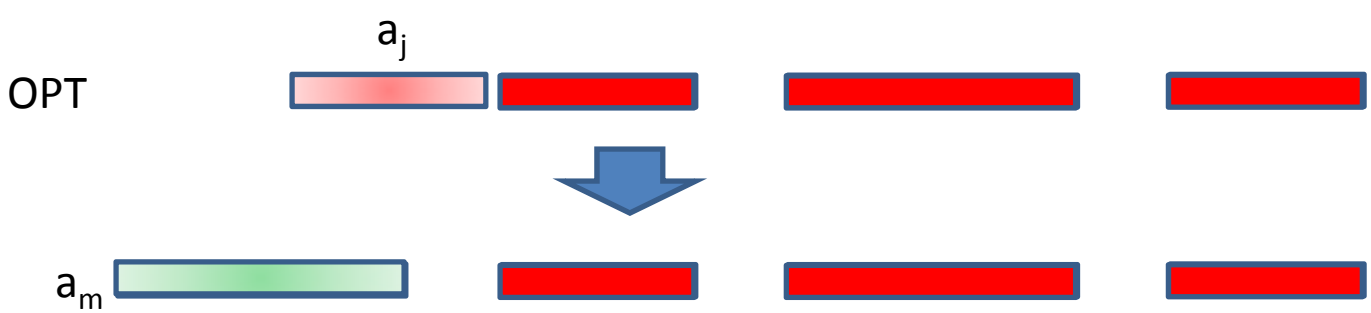
1. Find the optimal choice
2. Replace the optimal choice by the greedy choice
3. Show that the greedy choice is as good as the optimal choice

Greedy Choice

Consider a subproblem S_{ij} and
let a_m be the activity with the earliest finishing time

Lemma: There exists an optimal solution for S_{ij} that contains a_m; proof

- Let OPT be an optimal solution for S_{ij}
- Let a_j be the activity in OPT with the earliest finishing time
- If $a_m = a_j$, then the lemma is clearly true
- Otherwise, OPT-$\{a_j\} \cup \{a_m\}$ is <u>also</u> an optimal solution for S_{ij}



Greedy Algorithm for always select the last task to start

Note: We assume that the activities are already ordered in monotonic increasing finish time

```
Greedy-Activity-Selector(s[1..n], f[1..n])
B = {a_1}; m = 1; // greedily take activity 1
for k = 2 to n // for the other activities
   if f[k] ≤ s[m] then // we take it if not overlap
      B = B∪{a_k}; m = k;
return B;
```

Running time is clearly just: O(n)

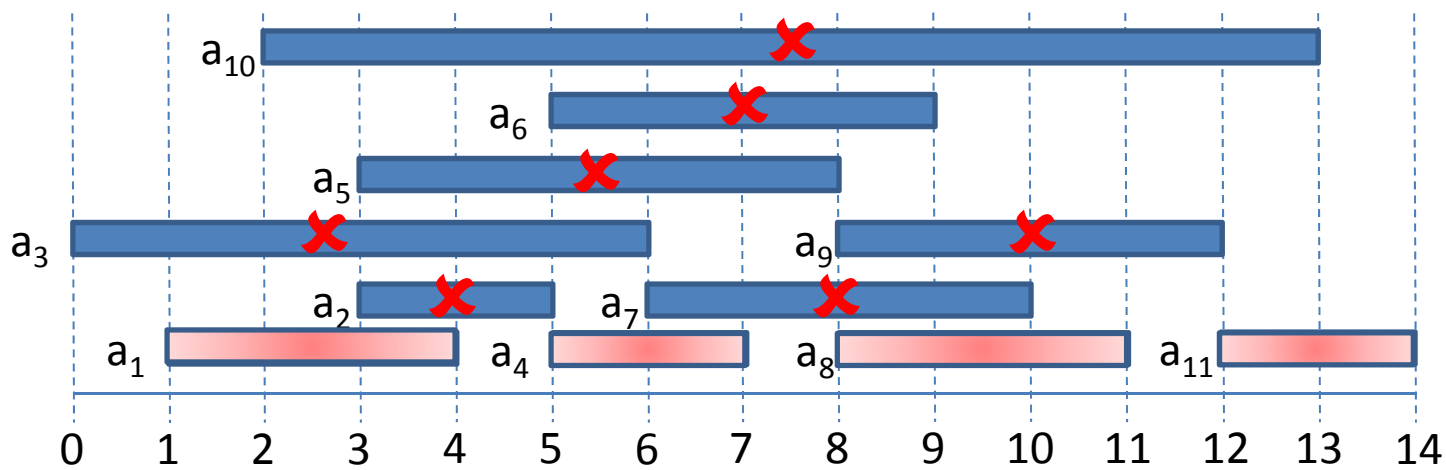
- Much faster than the $O(n^3)$ Dynamic Programming solution

Example

Input: A list of items already sorted by increasing finish time

Our greedy algorithm selects 4 activities: a_1, a_4, a_8, a_{11}

i	0	1	2	3	4	5	6	7	8	9	10	11	12
s_i	0	1	3	0	5	3	5	6	8	8	2	12	∞
f_i	0	4	5	6	7	8	9	10	11	12	13	14	∞



Greedy Algorithm Design Approach

1. Identify suitable definition for optimal substructures
2. Develop a recursive formulation based on optimal sub-structures
3. Develop a <u>greedy choice</u>
4. *Prove* that the greedy choice is one (of possibly many) optimal choice(s)*
5. Develop the (usually fast) greedy algorithm

*It is important NOT to forget about this step (else you cannot be sure if your greedy algorithm is optimal)

Are there more greedy choices?

- Yes. For example, we can choose the latest starting time.
  - We can use exchange argument to show that this strategy gives optimal solution.
  - Exercise!

Greedy Algorithm if we choose latest start activity first

Note: We assume that the activities are ordered in monotonic decreasing start time

```
Greedy-Activity-Selector(s[1..n], f[1..n])
B = {a1}; m = 1; // greedily take activity 1
for k = 2 to n // for the other activities
   if s[k] ≥ f[m] then // we take it if not overlap
      B = B ∪{ak}; m = k;
return B;
```

Running time is clearly just: O(n)

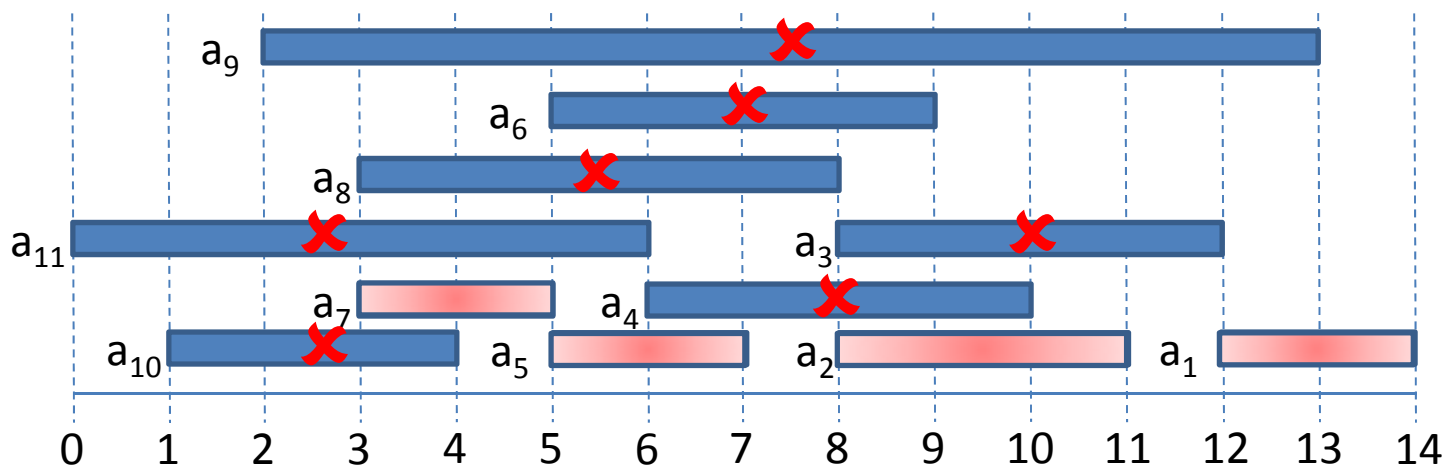
- Much faster than the $O(n^3)$ Dynamic Programming solution

Example

Input: A list of activities already sorted by decreasing start time

Our greedy algorithm selects 4 activities: a_1, a_2, a_5, a_7

i	0	1	2	3	4	5	6	7	8	9	10	11	12
s_i	0	12	8	8	6	5	5	3	3	2	1	0	∞
f_i	0	14	11	12	10	7	9	5	8	13	4	6	∞



Any other greedy choice?

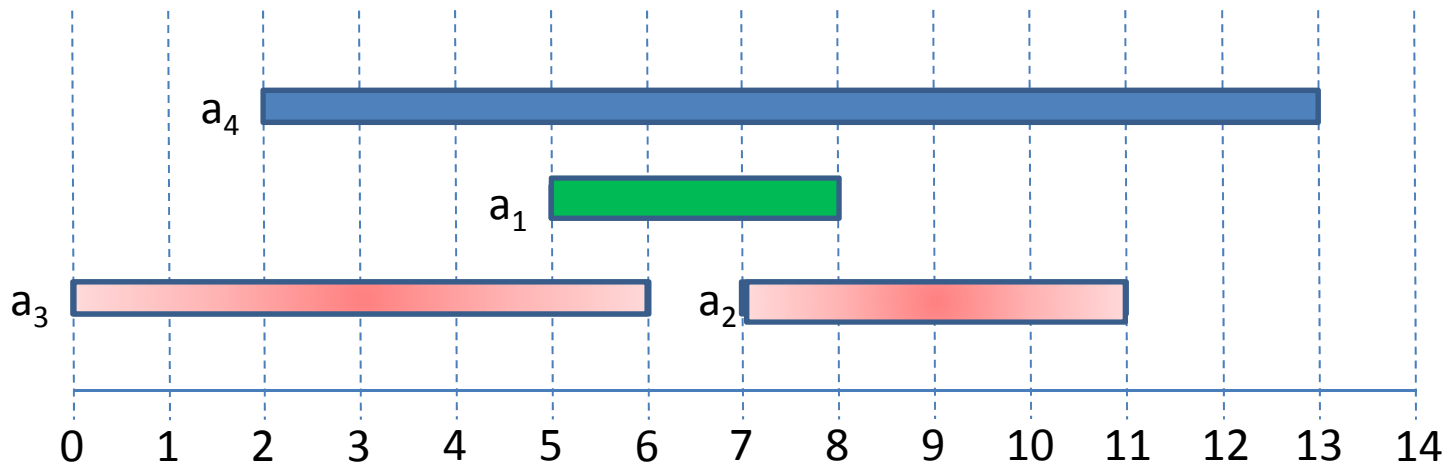
- Another example: We choose the shortest activity first.
  - This one looks good since short activity span shorter time.
  - However, this one may not give optimal solution.

Counter example to shortest activity first

- Below activities are sorted by their time spans.

i	1	2	3	4
s_i	5	7	0	2
f_i	8	11	6	13

- Greedy solution selects only one activity.
- Optimal solution selects 2 activities.



Huffman Code

Another Example

Representing a string using fixed-length code

Let S = "ababcaad", 8 characters

Fixed-length code to represent this sequence:

- The alphabet size is 4: {a, b, c, d}
- We can use 2 bits to represent each alphabet: {00, 01, 10, 11}
- In total, it takes 8*2 = 16 bits

a	b	a	b	c	a	a	d
00	01	00	01	10	01	01	11

Can we use <u>less space</u> to store the string S?

Variable-length code

Answer: <u>Variable-length code</u> may be shorter

- We can use shorter code to represent more frequent alphabet

However, we need space to separate codewords as the length of the code may be different…

- Solution: Prefix code
  - Codes in which no codeword is the prefix of some other codewords

Example: a=10, p=0, e=1101, l=111, i=1100

- Can you decode what is "10001111101"?

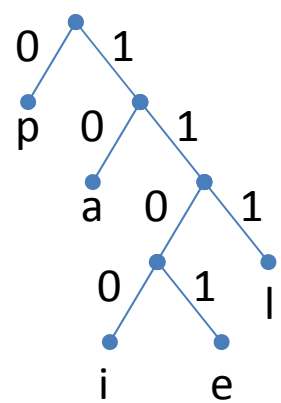
10-0-0-111-1101
Answer: <u>apple</u>

How to check if the code is a prefix code?

Answer: We can check the prefix property by drawing the code tree (for simplicity 0 on the left, 1 on the right)

- If every code is located at a <u>leaf in the code tree</u>, it is a prefix code

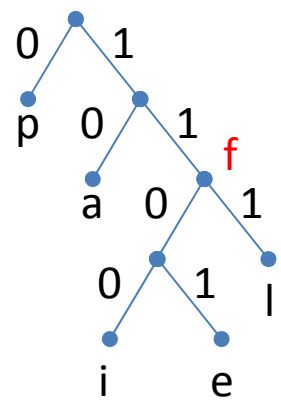
Example: a=10, p=0, e=1101, l=111, i=1100



Is the following code a prefix code?

a=10, p=0, e=1101, l=111, i=1100, f=11

Answer: No, this is not a prefix code since f is an internal node

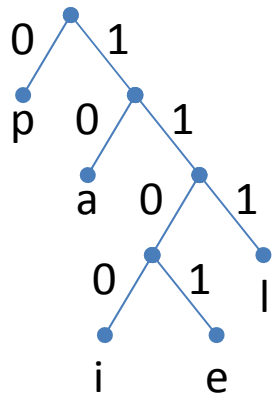


Decoding prefix code using code tree

Given the code tree,
we can decode a binary sequence in O(n) time

Example: 10001111101

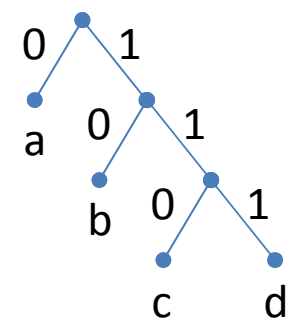
a p p l e



Example

We can use prefix code to shorten the encoding, e.g. we go back to the first example string S = "ababcaad"

- Recall that fixed-size code uses 16 bits



If we set a=0, b=10, c=110 and d=111, then we can encode S using just 14 bits

- In fact, this is the optimal solution
- Notice that more frequent letters usually have shorter code but less frequent letters may have longer code

a	b	a	b	c	a	a	d
0	10	0	10	110	0	0	111

Computational Problem (Draft)

Now consider a general string S, can we find the <u>shortest prefix code</u> to represent S?

Answer: We use <u>Huffman code</u>

Notations (1)

Given a string S,
let f(c) be the frequency of the character c in S

Example, for S = "ababcaad",
we have f(a) = 4, f(b) = 2, f(c) = 1, and f(d) = 1

Notations (2)

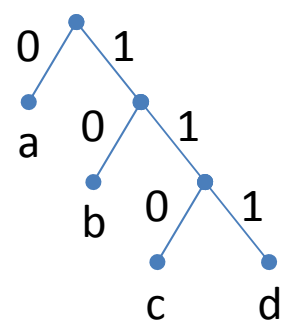
Let T be the code tree corresponding to a prefix code

For each character c, let d(c) be the depth of c in T

- Note: d(c) is also the length of the prefix code for c

Example: a = 0, b = 10, c = 110 and d = 111

- d(a) = 1, d(b) = 2, d(c) = 3, and d(d) = 3

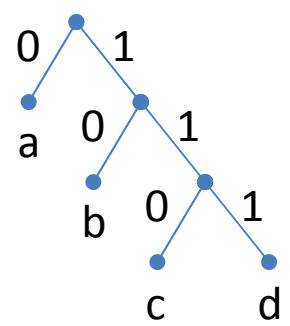


Encoding size B(T)

To encode S by the code tree T,
the total number of bits is B(T)= $\sum_{c \in C} f(c)d(c)$

Example:

- For S = "ababcaad“ and f(a) = 4, f(b) = 2, f(c) = 1, f(d) = 1
- For the code tree T and d(a) = 1, d(b) = 2, d(c) = 3, d(d) = 3
- The size of the encoding B(T) = 4*1 + 2*2 + 1*3 + 1*3 = 14



Computational Problem (Formal)

Consider a string S over an alphabet C

- Let f(x) be the number of occurrences of x in S

We aim to find OPT(C), that is,
the code tree T that minimizes B(T) $= \sum_{c \in C} f(c) d(c)$

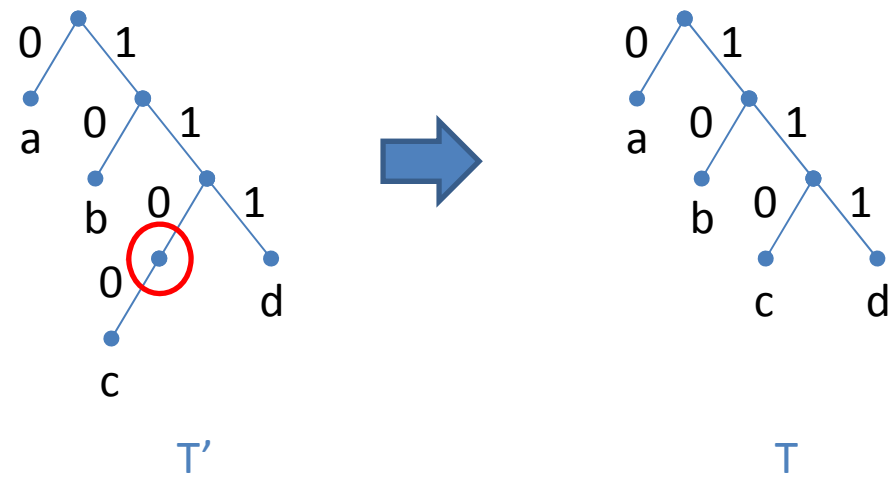
Observation

The optimal code tree must be a *full* binary tree

(full: every vertex other than the leaves has two children)

If the code tree T' is not a full binary tree, then we can get a better code tree T such that B(T) < B(T') by contracting the vertex with one child.

E.g.



1. Showing Optimal Substructure

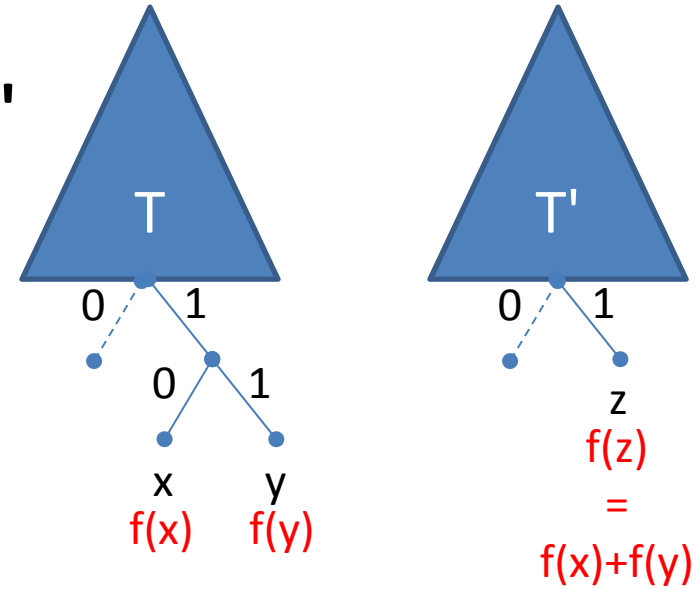
Let **T** be the optimal code tree for a string **S**

- Since **T** is a full binary tree, there exist two leaves **x** and **y** which are sibling at the deepest position

Let us replace both **x** and **y** with another character **z** and declare the resulting string to be **S'**

Let **T'** be the optimal code tree for **S'**

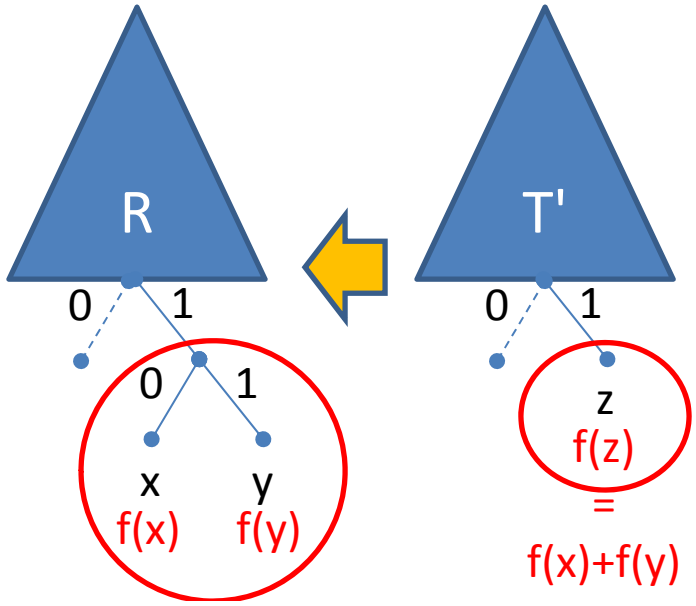
We claim: **B(T) = B(T')+f(x)+f(y)**



B(T) = B(T')+f(x)+f(y)

Proof: (←) Replace **z** in **T'** by a tree with two leaves **x** and **y**; then denote the tree to be **R**

B(R) = B(T')+f(x)+f(y) ≥ B(T)



$B(T) = B(T')+f(x)+f(y)$

Proof: (→) Replace the subtree with **x** and **y** in **T** by **z**

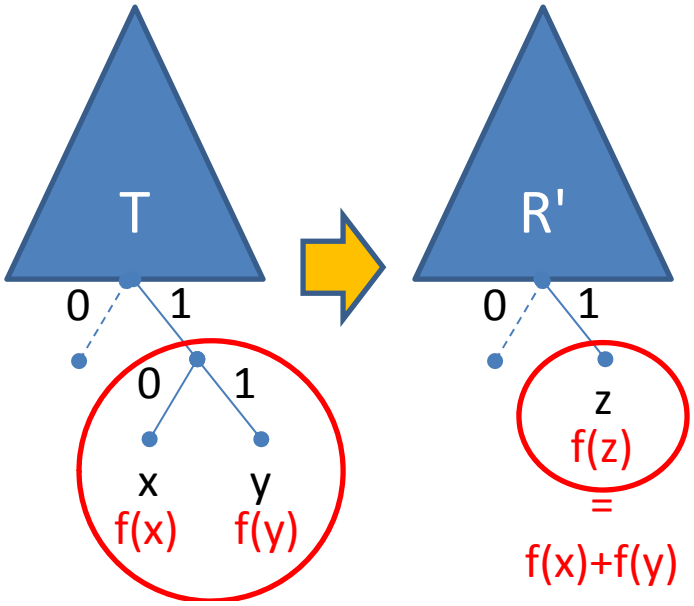
Let the tree be **R'**

$\mathbf{B(R') = B(T)-f(x)-f(y) \geq B(T')}$

Thus, $\mathbf{B(T')+f(x)+f(y) \leq B(T)}$

Hence, the claim is true:

$\mathbf{B(T) = B(T')+f(x)+f(y)}$



2. Making (+ Proving) Greedy Choice

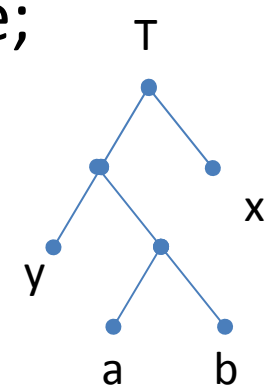
Let **x** and **y** be the two symbols that have the **lowest** and the **second lowest** frequencies, respectively

- Greedy intuition: Put them deepest in the code tree…

Lemma: There exists an optimal solution where the codewords for **x** and **y** have the same length (longest) and differ only in the last bit

Proof: Let **T** be an arbitrary optimal code tree; let **a** and **b** be the two symbols currently with the longest codewords

- Since **x** and **y** have the lowest frequencies, we have **f(x) ≤ f(a)** and **f(y) ≤ f(b)**



By swapping **x** with **a**, we get the tree **T'**

B(T)-B(T') = $\sum_{c \in C} f(c) d_T(c)$ - $\sum_{c \in C} f(c) d_{T'}(c)$ = // (def)

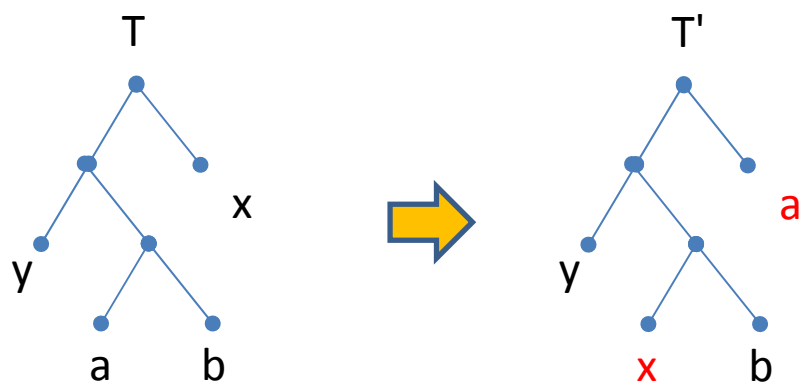
$f(x)d_T(x) + f(a)d_T(a) - f(x)d_{T'}(x) - f(a)d_{T'}(a)$ = // impt ones

$f(x)d_T(x) + f(a)d_T(a) - f(x)d_T(a) - f(a)d_T(x)$ = // substitution

$(f(a)-f(x)) * (d_T(a)-d_T(x))$ // rearrange the equations

≥ 0 // both terms above are positive!

Hence, $B(T) \geq B(T')$



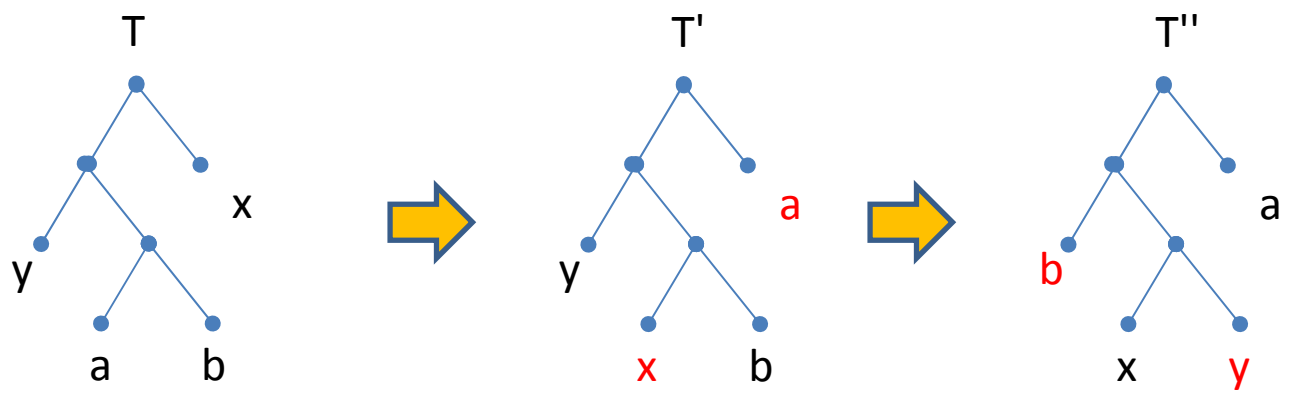
By swapping **y** with **b**, we get the tree **T''**

With the same argument as in the previous slide,
we can show $B(T') \geq B(T'')$

We now have $B(T) \geq B(T') \geq B(T'')$, or $B(T) \geq B(T'')$

As T is an arbitrary optimal tree, we also have
$B(T) \leq B(T') \leq B(T'')$, or $B(T) \leq B(T'')$

We conclude that $B(T) = B(T'')$, i.e. T'' is also an optimal tree
in which x and y appear as sibling leaves of maximum depth



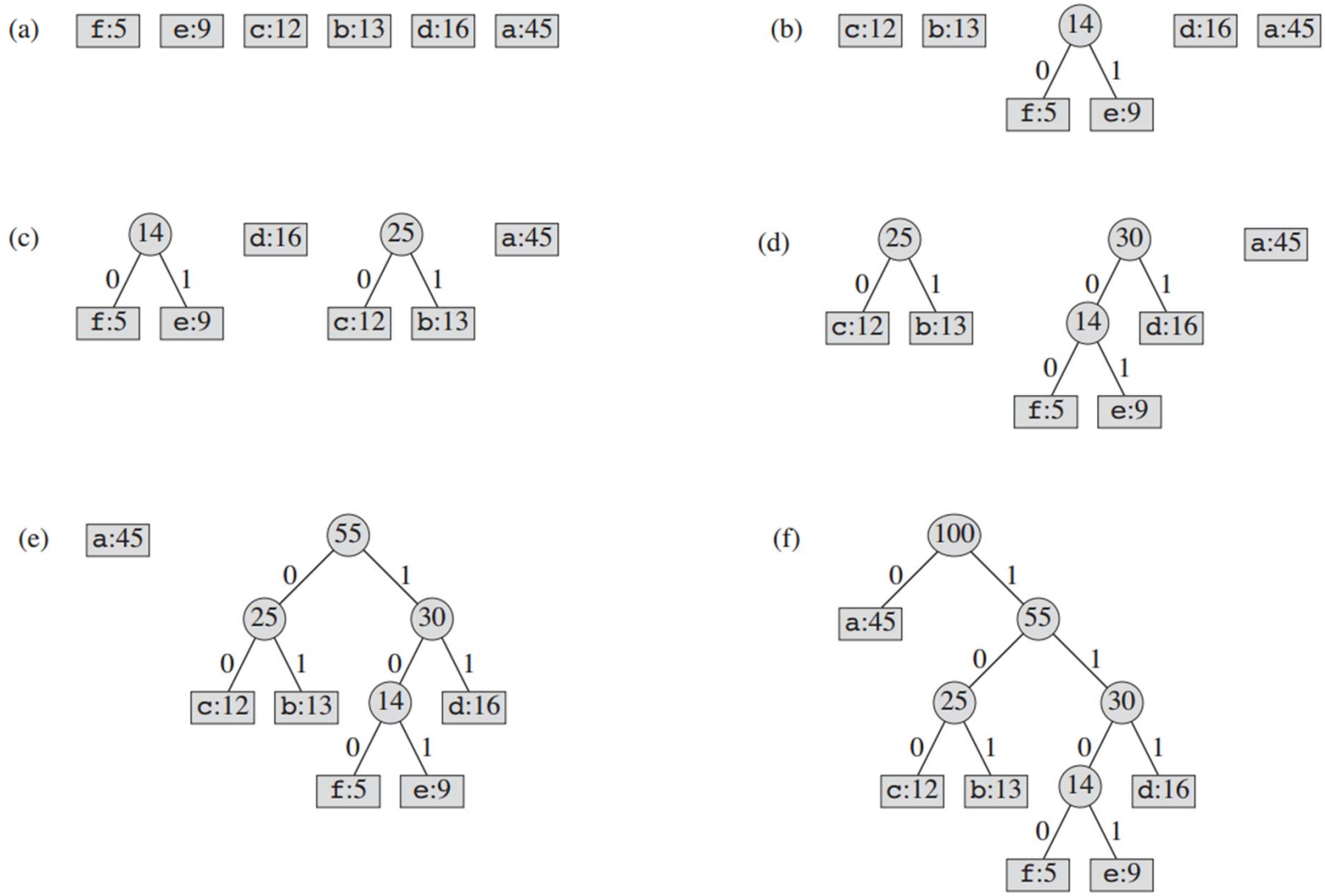
Huffman Greedy Algorithm

Consider a string **S** over an alphabet **C**
and **f(x)** be the number of occurrences of **x** in **S**

```
Huffman(C)
N = |C|;
Q = C;
for i = 1 to n-1 // O(n log n) algorithm
  x = Extract-Min(Q); y = Extract-Min(Q); // O(log n)
  create a new node z and set x and y be its children;
  f(z) = f(x)+f(y);
  insert(Q, z);
return Extract-Min(Q); // root of the optimal code tree
```

This optimal code tree is also called the **Huffman tree**

Execution Example



Nearest Neighbor Heuristic for TSP

Greedy heuristic (may be sub-optimal) applied to the optimization version of an NP-Complete problem

Nearest Neighbor Heuristic for TSP

We know that TSP is an NPC problem (Lecture 08)

We have seen the $O(2^n * n^2)$ Held & Karp's DP for TSP which have exponential time complexity (Lecture 10)

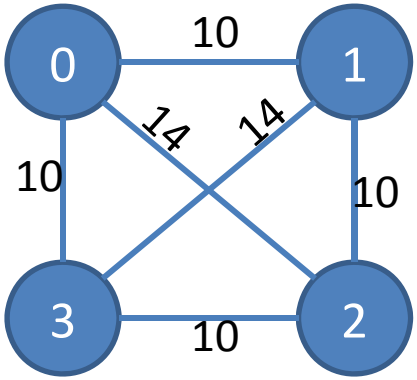
If we can sacrifice the need of reaching optimal solution, can we obtain "good" enough solution for <u>*some*</u> instances of TSP?

Nearest Neighbor Heuristic (NNH)

```
NNH(G(V, E)) // heuristic = a search strategy
  cur_vertex = 0;
  visited[cur_vertex] = true;
  cost = 0;
  while (not all vertices are visited)
    "greedily" select the shortest edge e connecting
    the cur_vertex u with an unvisited vertex v;
    // we do NOT have a proof for this greedy choice
    cost = cost + w(e);
    cur_vertex = v;
    visited[cur_vertex] = true;

  output cost+w(cur_vertex, 0); // not necessarily
optimal
```

“Perfect” Instance for NNH



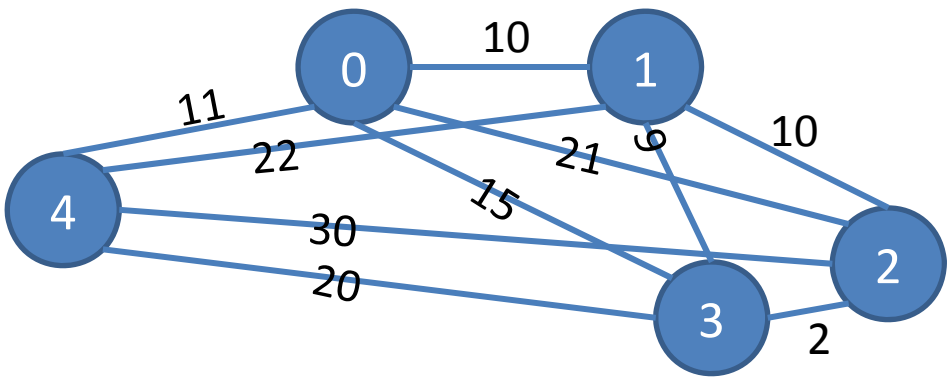
(near) Euclidean instances, weight of edge 0-2 and 1-3 are rounded to nearest integer

For this one, NNH produces tour 0-1-2-3 (or 0-3-2-1), both with optimal cost = 40

This is just a “lucky” case though…

“Good” Instance for NNH

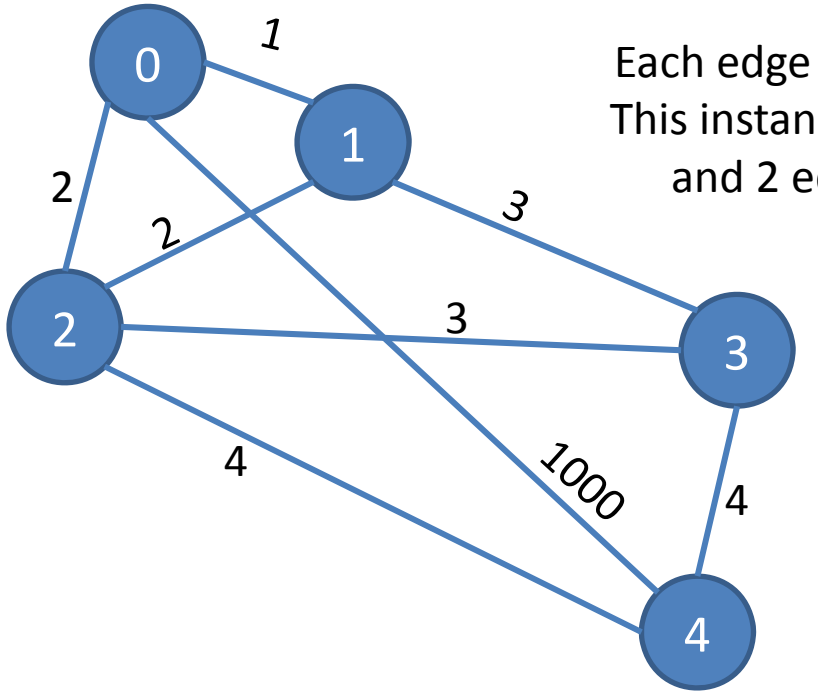
(near) Euclidean instances



The optimal tour is 0-1-2-3-4, cost 10+10+2+20+11 = 53

But for this instance, NNH produces tour 0-1-3-2-4 with cost 10+9+2+30+11 = 62, which is ~17% worse

“Bad” Instance for NNH



Each edge weight is in **Billion**
This instance is **<u>not Euclidean</u>**
and 2 edges not shown

The optimal tour is 0-1-3-4-2, cost 1+3+4+4+2 = 14B

But for this instance, NNH produces tour 0-1-2-3-4 with cost 1+2+3+4+1000 = 1010B, which is ~7100% :O worse

Pretty bad heuristic?…

We classify this solution as greedy *heuristic* and <u>not</u> the proper greedy algorithms as it does not has a correct greedy choice proof (otherwise P = NP)

On Euclidean TSP instances, this NN heuristic is not so bad (more details on other approximation algorithms for TSP with proper bounds in <u>the next Lecture 12</u>)

On non Euclidean TSP instances, it can be really bad…

Should we avoid using it? (open-ended discussion)

Summary (1)

Today, we have seen more examples of “classic” greedy algorithms on top of greedy algorithms that you have seen in CS2010/CS2020, i.e. Prim’s, Kruskal’s, Dijkstra’s

We **<u>may</u>** be able to use Greedy algorithm if:

1. The problem has optimal substructure
2. There is a proven greedy choice

Summary (2)

We **<u>may</u>** be able to use Greedy algorithm if:

1. The problem has optimal substructure
   - Prim's/Kruskal's: After we greedily take an edge with min weight, we are left with the same MST problem, with one less edge
   - Dijkstra's: After we greedily take a vertex with min distance estimate, we are left with the same SSSP problem, with one less vertex
2. There is a proven greedy choice
   - Prim's/Kruskal's: Proof in Section 23.1 of CLRS
   - Dijkstra's: Proof in Section 24.3 of CLRS

Summary (3)

When a certain **P** problem is solvable with greedy algorithm, the solution is often fast (usually iterative)

- If greedy is not applicable for that problem, we may use the "slower", but still polynomial solutions, like Dynamic Programming, Divide and Conquer, or even Brute Force
- But for some time critical applications, we may still use greedy algorithm on P problems (even if it has no proof of correctness)

On **NPC** problems, there will not be any correct greedy algorithm unless P = NP, so we call them: greedy heuristics

- If we can afford to obtain non optimal solution as long as it can be generated quickly, this greedy heuristics may still be applicable